

Documentación Técnica

Prueba de Desarrollador

BPOAmericas.TestDeveloper

Stack utilizado: .NET 8 (LTS), ASP.NET Core Web API, ASP.NET Core MVC, JWT Bearer, Swagger

Sebastian Acero León
29 de Septiembre del 2026

Contenido

Creacion del repositorio	3
Clonación del repositorio.....	4
Creación del proyecto en visual studio 2022.....	5
Instalación nugget JWTBearer	7
API - DTOs	8
API - Application.....	9
API - API.....	13
API - Swagger	14
Postman	16
MVC - Objetos	18
MVC – Servicios.....	19
MVC - Controlador	20
MVC – VISTA	21
Vista del proyecto terminado	22
Subir cambios al repositorio	24

Creacion del repositorio

Para la implementación de la prueba técnica se empezó con la creación del repositorio en GITHUB, dentro de mi repositorio como regla, tengo que la empresa es la guía, el segundo es lo que se va a desarrollar y por ultimo el modulo, por ese motivo se crea de la siguiente manera

New repository

Create a new repository

Repositories contain a project's files and version history. Have a project elsewhere? [Import a repository](#).
Required fields are marked with an asterisk (*).

1 General

Owner * **Repository name ***

Walas11 / BPOAmericas.TestDeveloper.API

✓ BPOAmericas.TestDeveloper.API. is available.

Great repository names are short and memorable. How about [literate-waddle?](#)

Description

0 / 350 characters

2 Configuration

Choose visibility *
Choose who can see and commit to this repository

Public

Add README
READMEs can be used as longer descriptions. [About READMEs](#)

Off

Add .gitignore
.gitignore tells git which files not to track. [About ignoring files](#)

VisualStudio

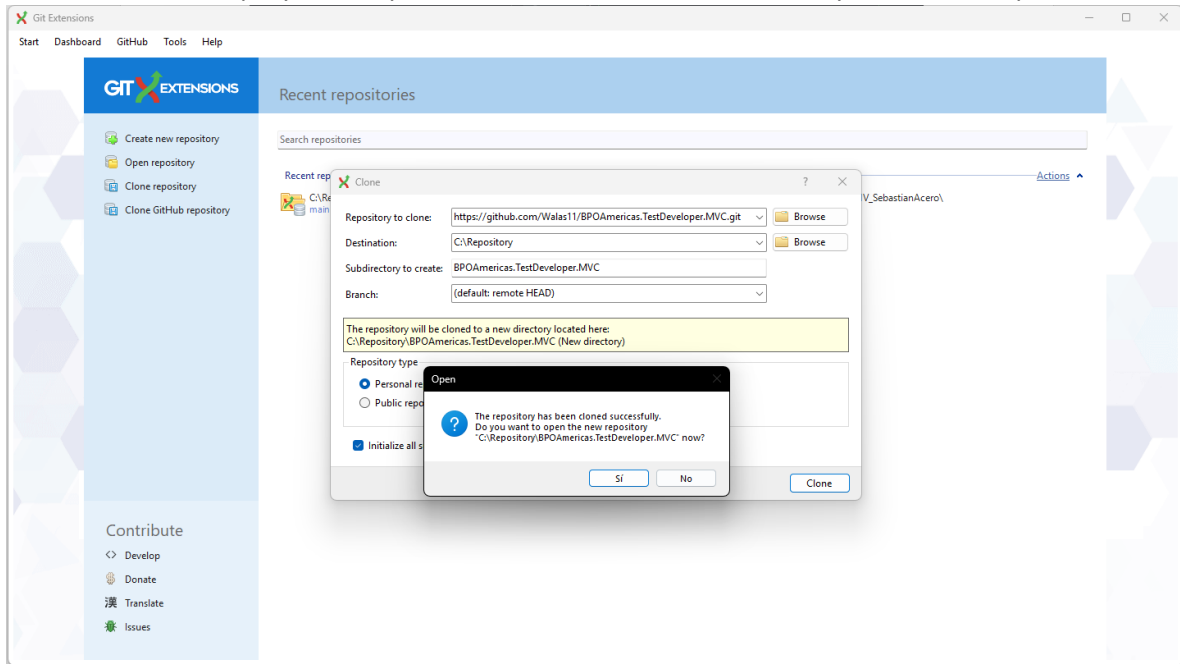
Add license
Licenses explain how others can use your code. [About licenses](#)

No license

Create repository

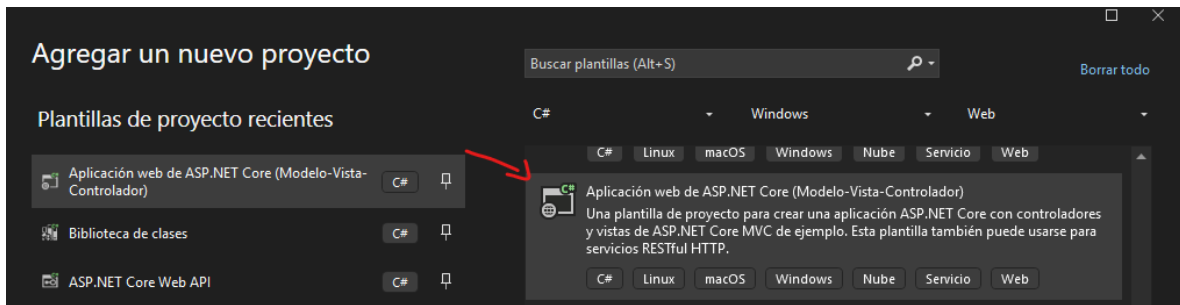
Clonación del repositorio

Con la creación del proyecto, copiamos su URL en la extensión de Git y creamos el repositorio local



Creación del proyecto en visual studio 2022

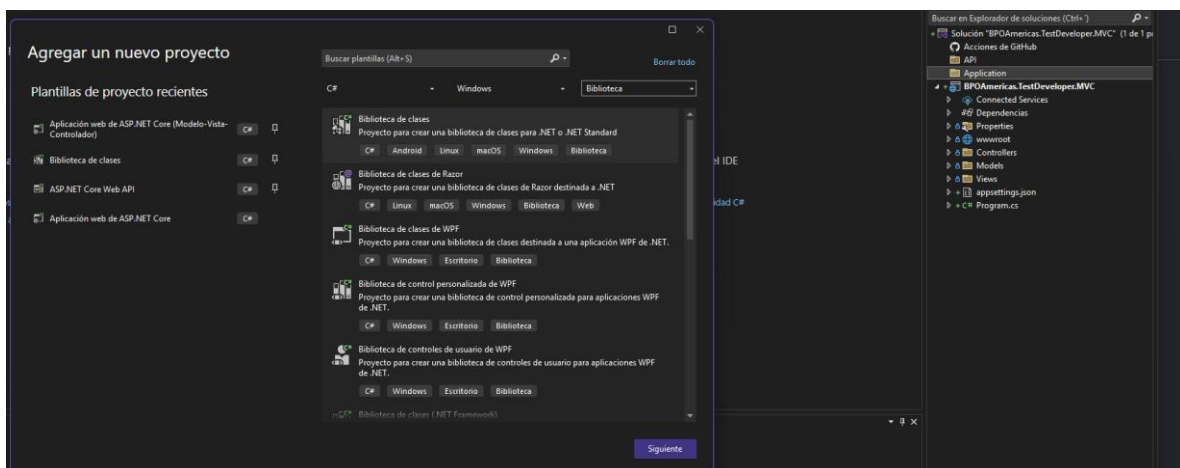
Con la creación del repositorio abrimos visual studio 2022 y nos vamos al apartado de C#, Windows y Web en los filtros, con el fin de encontrar el proyecto Aplicación web de ASP.NET Core (Modelo-Vista-Controlador), se selecciona este proyecto a la interpretación de “Solución de visual studio con el proyecto WEB API y componentes desarrollados” en la parte de MVC se encontrará la parte visual del desarrollo



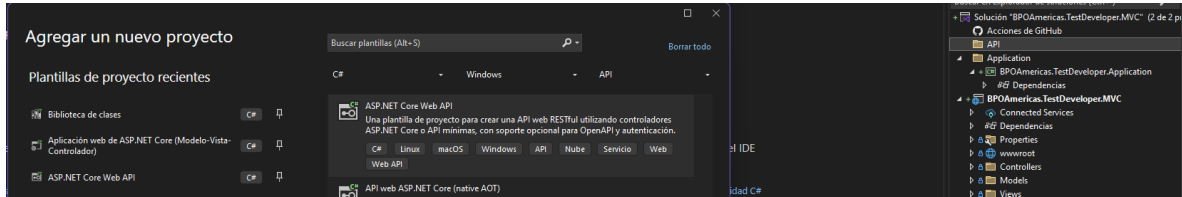
Creamos las carpetas con las que se va a estructurar la arquitectura en capas, teniendo las capas de:

- API
- Aplicación
- Comun
- MVC

Esta estructura hace más cómodo manejar la responsabilidad única y la inyección de dependencias, ya que no tendremos interfaces gigantes, sino únicamente las interfaces necesarias para cada servicio. Aplicación es una biblioteca de clase como se muestra acontinuacion

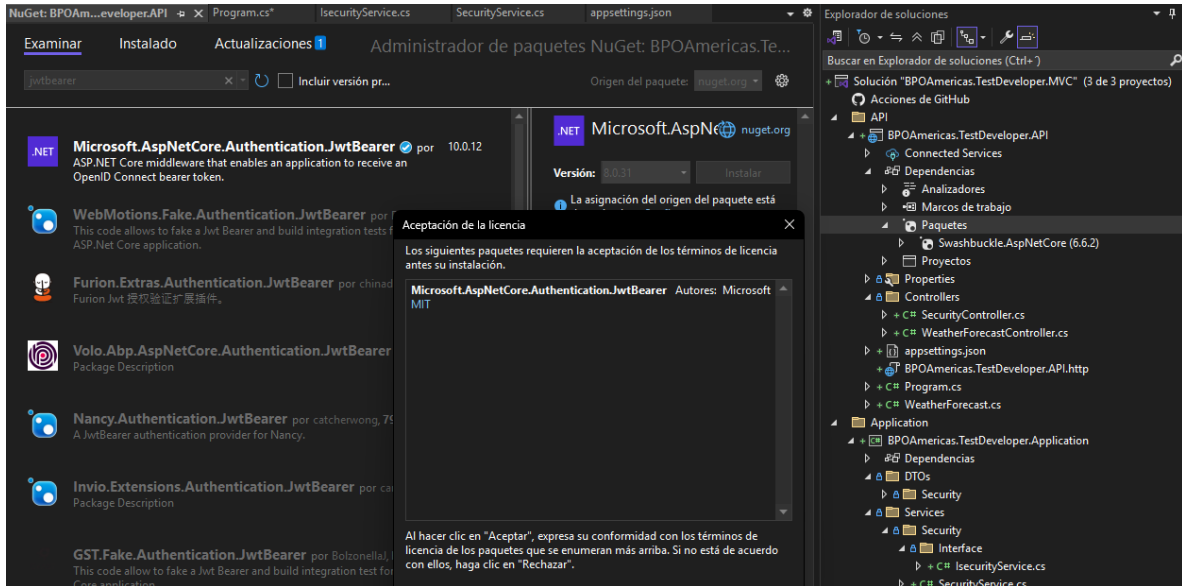


Api como su nombre lo dice es donde se van a recibir las peticiones REST, en este caso solo una la petición de seguridad, por eso creamos un proyecto API CORE



Instalación nugget JWTBearer

Los Nuggets son importantes ya que debemos implementar JWT_Bearer una seguridad por tokens, que sin duda se necesita para implementar la seguridad por tokens, agregamos e instalamos la versión mas reciente a la versión del API en este caso 8.0.31



API - DTOs

Lo primero a implementar son los DTOs

Ya que estos serán la salida y entrada de los controladores y los servicios, son nuestros objetos con los que vamos a tener entre el código, en este caso no se implemento base de datos, por lo que no hay Entitys pero estos dos harán de objetos y entidades para la respuesta y la entrega de información

```
namespace BPOAmericas.TestDeveloper.Application.DTOs.Security
{
    3 referencias
    public class LoginUserRequestDto
    {
        1 referencia
        public string Username { get; set; }
        1 referencia
        public string UserPassword { get; set; }
        0 referencias
        public string ClientIP { get; set; }
        0 referencias
        public string UserAgent { get; set; }
    }
}
```

```
namespace BPOAmericas.TestDeveloper.Application.DTOs.Security
{
    public class LoginUserResponseDto
    {
        public int IdUser { get; set; }
        public string? Username { get; set; }
        public string? UserProfile { get; set; }
        public DateTime LastLoginDate { get; set; }
    }
}
```

API - Application

Con la creación de objetos nos dirigimos a la creación del Token, en la cual por interfaces accedemos a ella

```
using BPOAmericas.TestDeveloper.Application.DTOs.Security;

namespace BPOAmericas.TestDeveloper.Application.Services.TokenService.Interface
{
    4 referencias
    public interface ITokenService
    {
        2 referencias
        string GetToken(LoginUserResponseDto responseDto);
    }
}
```

Se sigue la interpretación del código fuente de Microsoft para la creación del token bearer, retornándonos un token por debajo gracias al nugget instalado

```
TokenService.cs | IsecurityService.cs | AuthService.cs | SecurityService.cs* | TokenService.cs
BPOAmericas.TestDeveloper.Application | BPOAmericas.TestDeveloper.Application.Ser | GetToken(LoginUserResponseDto)

1  using BPOAmericas.TestDeveloper.Application.DTOS.Security;
2  using BPOAmericas.TestDeveloper.Application.Services.TokenService.Interface;
3  using Microsoft.IdentityModel.Tokens;
4  using System.IdentityModel.Tokens.Jwt;
5  using System.Security.Claims;
6  using System.Text;
7
8  namespace BPOAmericas.TestDeveloper.Application.Services.TokenService
9  {
10     1 referencia
11     public class TokenService : ITokenService
12     {
13         private readonly string _token = "BPOAmericas2026*ClaveSecretaJWT!";
14         2 referencias
15         public string GetToken(LoginUserResponseDto responseDto)
16         {
17             var claims = new[]
18             {
19                 new Claim("IdUser", responseDto.IdUser.ToString()),
20                 new Claim(ClaimTypes.Name, responseDto.UserName),
21                 new Claim("UserProfile", responseDto.UserProfile)
22             };
23
24             var key = new SymmetricSecurityKey(Encoding.UTF8.GetBytes(_token));
25             var creds = new SigningCredentials(key, SecurityAlgorithms.HmacSha256);
26
27             var token = new JwtSecurityToken(
28                 issuer: "BPOAmericas",
29                 audience: "BPOAmericas",
30                 claims: claims,
31                 expires: DateTime.Now.AddHours(1),
32                 signingCredentials: creds
33             );
34
35             return new JwtSecurityTokenHandler().WriteToken(token);
36         }
37     }
38 }
```

Con la implementación del Token hacemos la correspondiente interfaz del servicio de Seguridad

```
BPOAmericas.TestDeveloper.Application | BPOAmericas.TestDeveloper.Application.Ser | LoginUser(LoginUserRequestDto)

1  using BPOAmericas.TestDeveloper.Application.DTOS.Security;
2  using System;
3  using System.Collections.Generic;
4  using System.Linq;
5  using System.Text;
6  using System.Threading.Tasks;
7
8  namespace BPOAmericas.TestDeveloper.Application.Services.Security.Interface
9  {
10     4 referencias
11     public interface IsecurityService
12     {
13         2 referencias
14         LoginUserResponseDto LoginUser(LoginUserRequestDto loginUserRequest);
15     }
16 }
```

Con la interfaz creada, procedemos a quemar el usuario y la contraseña ya que no tenemos base de datos de la siguiente manera

```
UserName == "BPOAmericas@bpoamericas.com" && UserPassword == "BPOAmericas2026*"
```

y respondemos con un ok cuando en los parámetros del controlador vienen de la misma manera

```
using BPOAmericas.TestDeveloper.Application.DTOs.Security;
using BPOAmericas.TestDeveloper.Application.Services.Security.Interface;
using BPOAmericas.TestDeveloper.Application.Services.TokenService.Interface;
using BPOAmericas.TestDeveloper.Common.Services;

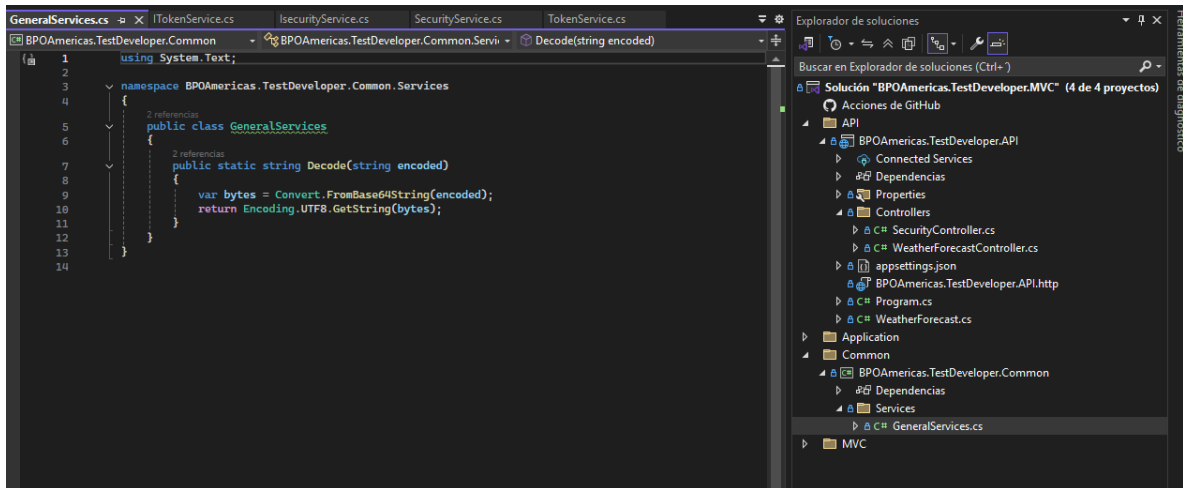
namespace BPOAmericas.TestDeveloper.Application.Services.Security
{
    1 referencia
    public class SecurityService : IsecurityService
    {
        2 referencias
        public LoginUserResponseDto LoginUser(LoginUserRequestDto requestDto)
        {
            string UserName = GeneralServices.Decode(requestDto.UserName);
            string UserPassword = GeneralServices.Decode(requestDto.UserPassword);

            if (UserName == "BPOAmericas@bpoamericas.com" && UserPassword == "BPOAmericas2026*")
            {
                var response = new LoginUserResponseDto
                {
                    IdUser = 5,
                    UserName = "TEST USER",
                    UserProfile = "lowlevel",
                    LastLoginDate = DateTime.Now
                };

                return response;
            }

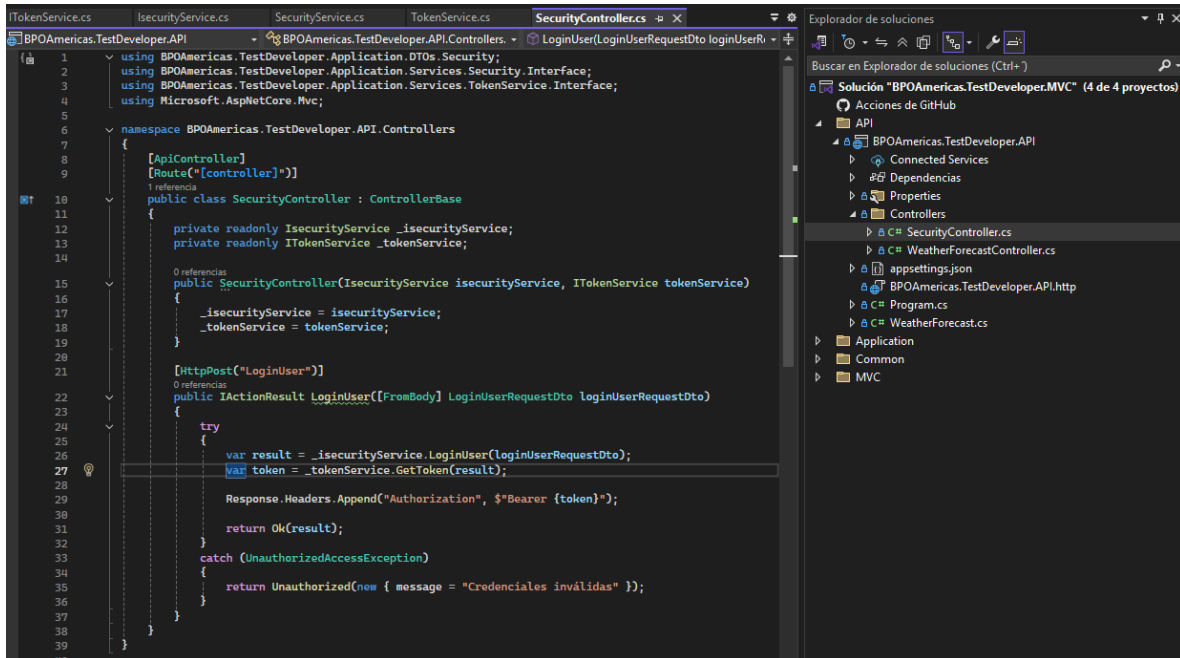
            throw new UnauthorizedAccessException("Usuario o Contraseña incorrecta");
        }
    }
}
```

NOTA: Se crea un GeneralService con el fin de tener la codificación aparte del servicio, ya que esto no corresponde a el servicio de login sino a un servicio general, no se vuelva hacer en ningún método nuevamente y siempre se use la misma codificación



API - API

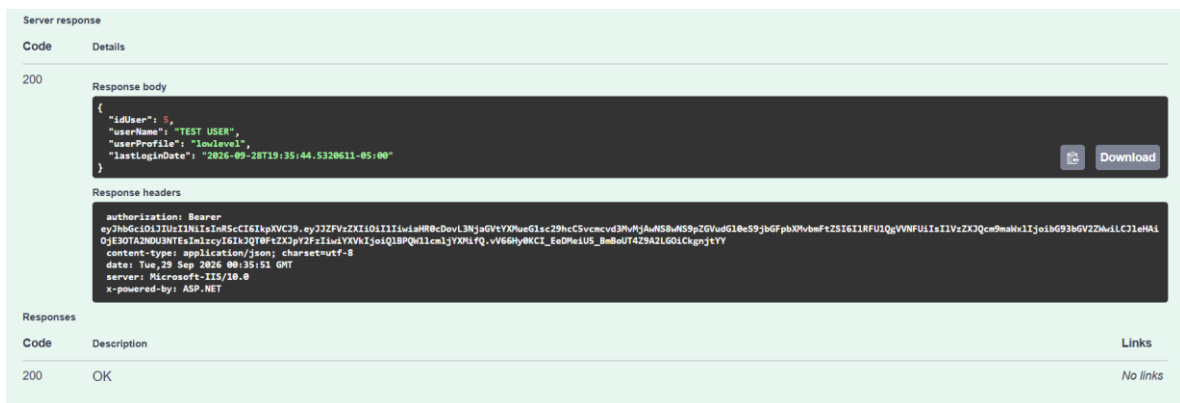
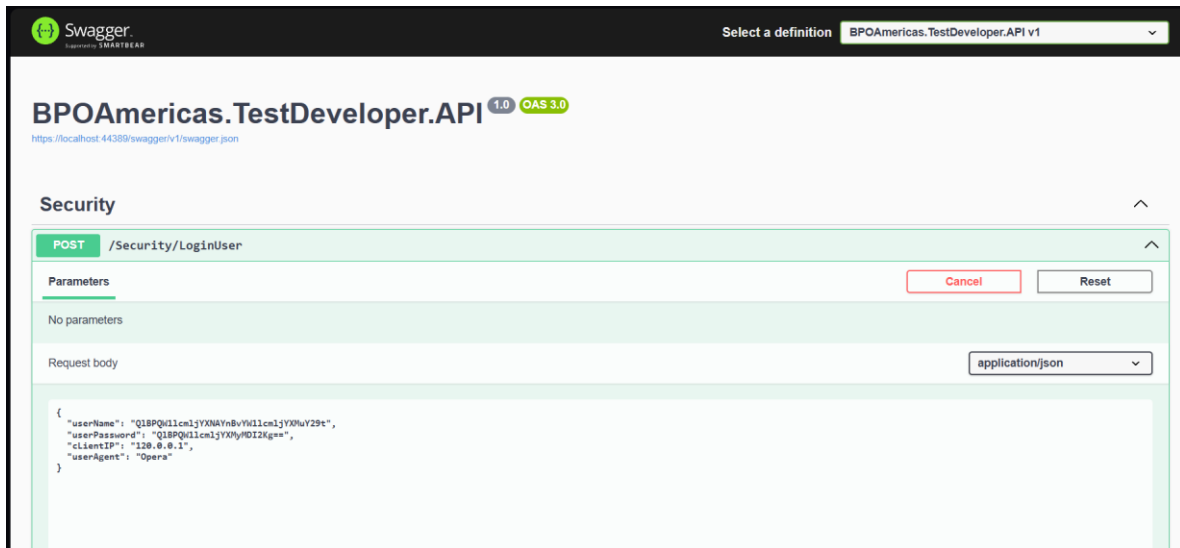
Con nuestros servicios creados podemos continuar con el controlador instanciamos los servicios ya que desde este vamos a acceder con una respuesta ok, instanciamos Token y Seguridad, obtenemos los resultados y generamos el token bearer de ultimas ya que este mismo se enviara por debajo para la aceptación en el MVC que es el Front



```
1 using BPOAmericas.TestDeveloper.Application.DTOS.Security;
2 using BPOAmericas.TestDeveloper.Application.Services.Security.Interface;
3 using BPOAmericas.TestDeveloper.Application.Services.TokenService.Interface;
4 using Microsoft.AspNetCore.Mvc;
5
6 namespace BPOAmericas.TestDeveloper.API.Controllers
7 {
8     [ApiController]
9     [Route("api/[controller]")]
10    public class SecurityController : ControllerBase
11    {
12        private readonly ISecurityService _isecurityService;
13        private readonly ITokenService _tokenService;
14
15        public SecurityController(ISecurityService isecurityService, ITokenService tokenService)
16        {
17            _isecurityService = isecurityService;
18            _tokenService = tokenService;
19        }
20
21        [HttpPost("LoginUser")]
22        public IActionResult LoginUser([FromBody] LoginUserRequestDto loginUserRequestDto)
23        {
24            try
25            {
26                var result = _isecurityService.LoginUser(loginUserRequestDto);
27                var token = _tokenService.GetToken(result);
28
29                Response.Headers.Append("Authorization", $"Bearer {token}");
30
31                return Ok(result);
32            }
33            catch (UnauthorizedAccessException)
34            {
35                return Unauthorized(new { message = "Credenciales inválidas" });
36            }
37        }
38    }
39 }
```

API - Swagger

Como resultado tenemos swagger que está implícito en .Net 8, para poder hacer pruebas de que el código quedó bien implementado



Se hace la prueba con el siguiente JSON

```
{
  "userName": "QIBPQW1lcmljYXNAYnBvYWw1lcmljYXMuY29t",
  "userPassword": "QIBPQW1lcmljYXMMyMDI2Kg==",
  "cLientIP": "120.0.0.1",
  "userAgent": "Opera"
}
```

Se obtiene el userName y userPassword con una herramienta web de codificación

Codifique en formato Base64

Simplemente introduzca los datos y pulse el botón de codificar.

BPOAmericas2026*

● Para codificar binarios (como imágenes, documentos, etc.) utilice el formulario de carga de archivos que encontrará un poco más abajo en esta página.

UTF-8

Conjunto de caracteres de destino.

LF (Unix)

Separador de nueva línea de destino.

☐ Codifique cada línea por separado (útil cuando tiene varias entradas).

☐ Divida las líneas en trozos de 76 caracteres de largo (útil para MIME).

☐ Realice una codificación URL segura (utiliza el formato Base64URL).

 Modo en directo DESACTIVADO

Codifica en tiempo real mientras escribe o pega (sólo admite el juego de caracteres UTF-8).

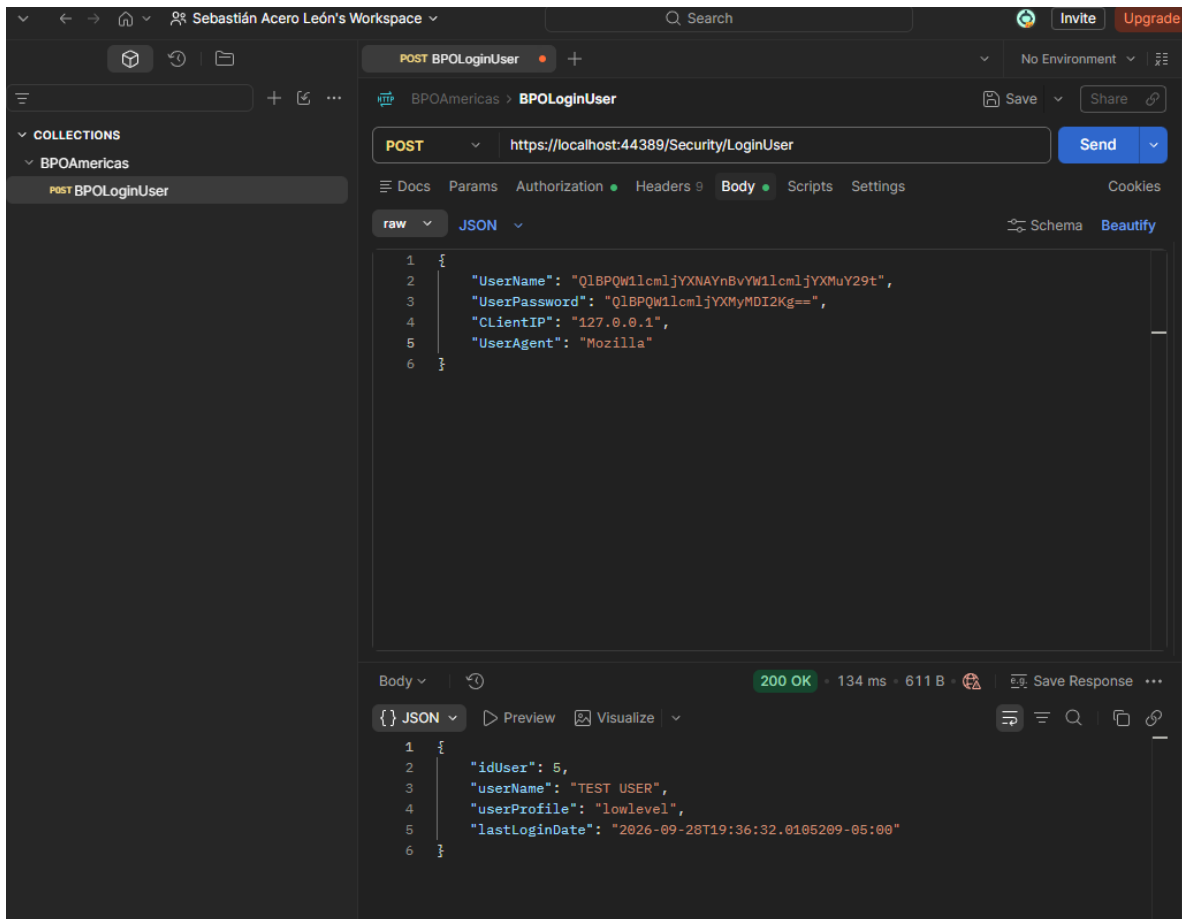
> CODIFICAR <

Codifica sus datos en la zona de abajo.

QIBPQW1lcmJlYXMyMDI2Kg==

Postman

Hacemos la prueba en POSTMAN igual que la imagen de referencia:



Obtenemos un código 200 que es ok en todas sus interpretaciones, en el nuevo postman este guarda el bearer token de una manera intrínseca ya que es un token y por eso se ve así en la siguiente imagen:

POST BPOLoginUser

No Environment

BPOAmericas > BPOLoginUser

Save

Share

POST

https://localhost:44389/Security/LoginUser

Send

Docs

Params

Authorization

Headers 9

Body

Scripts

Settings

Cookies

Auth type

Bearer Token

The authorization header will be automatically generated when you send the request. Learn more about [authorization](#)

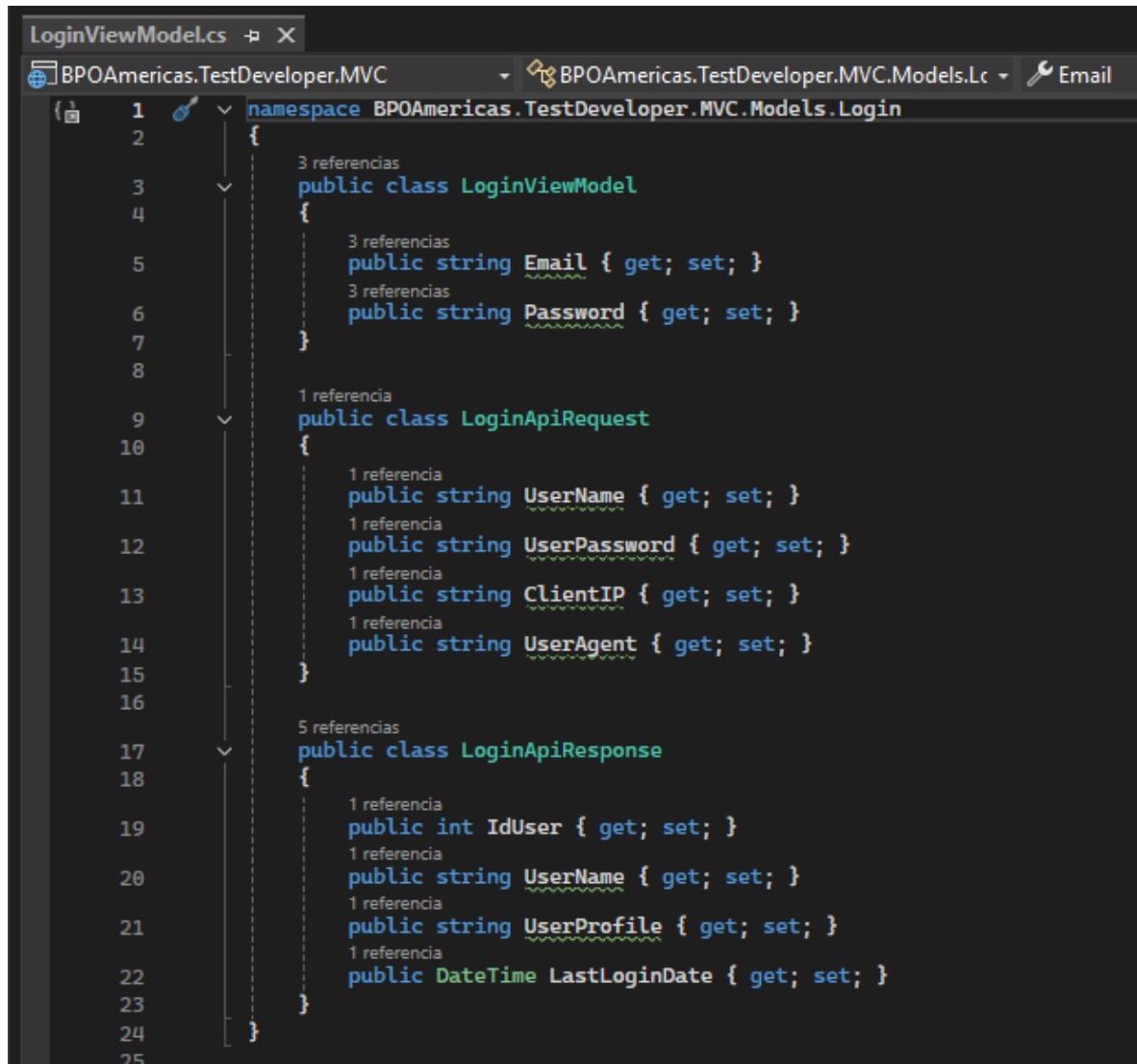
Bearer Token

Token

{{bearer_token_0egi}}

MVC - Objetos

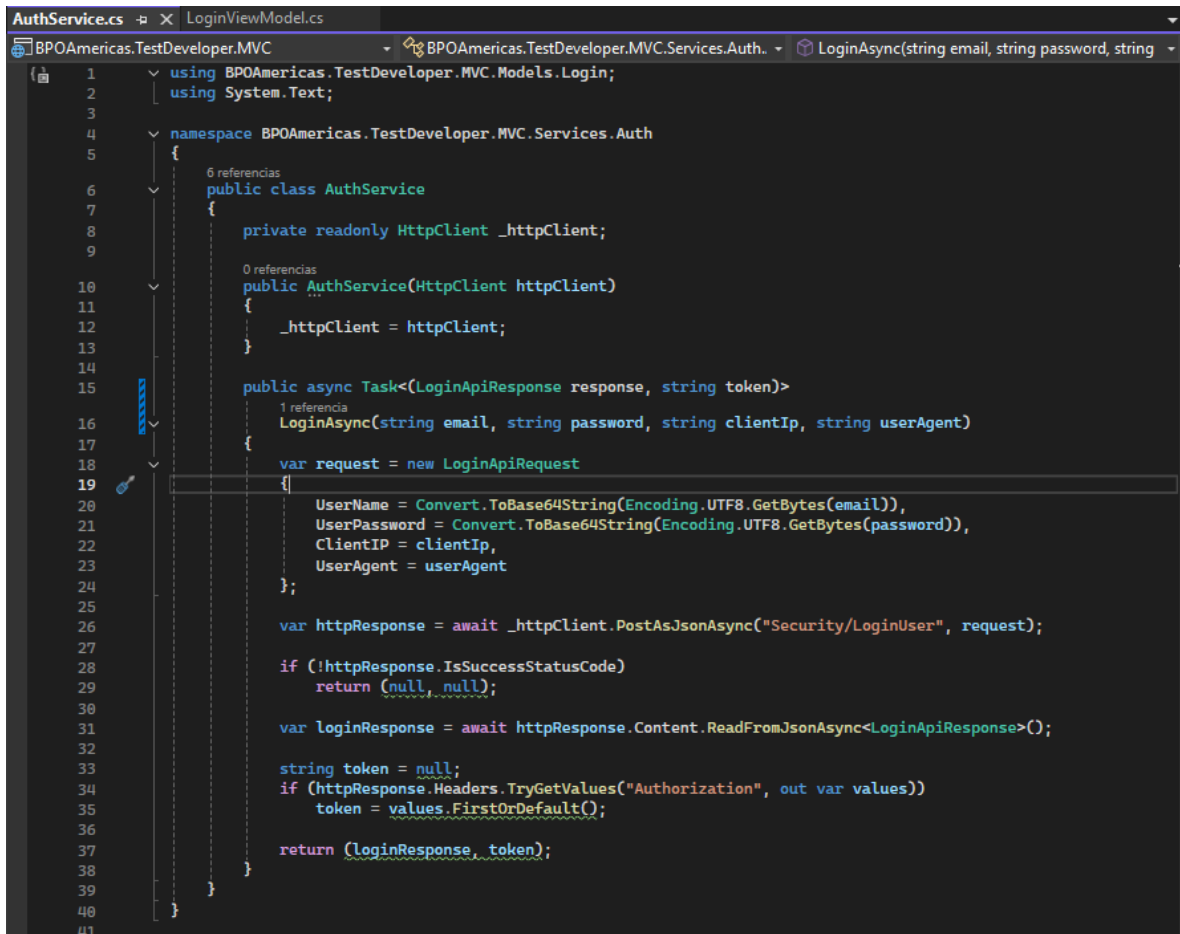
Con el API funcionando se realiza el tema Front, con Blazor ya que el entregable es un único proyecto “Solución de visual studio con el proyecto WEB API y componentes desarrollados”, se inicia con el tema de los modelos, los que veremos en pantalla y los mismos del API



```
1 namespace BPOAmericas.TestDeveloper.MVC.Models.Login
2 {
3     3 referencias
4     public class LoginViewModel
5     {
6         3 referencias
7         public string Email { get; set; }
8         3 referencias
9         public string Password { get; set; }
10    }
11
12    1 referencia
13    public class LoginApiRequest
14    {
15        1 referencia
16        public string UserName { get; set; }
17        1 referencia
18        public string UserPassword { get; set; }
19        1 referencia
20        public string ClientIP { get; set; }
21        1 referencia
22        public string UserAgent { get; set; }
23    }
24
25    5 referencias
26    public class LoginApiResponse
27    {
28        1 referencia
29        public int IdUser { get; set; }
30        1 referencia
31        public string UserName { get; set; }
32        1 referencia
33        public string UserProfile { get; set; }
34        1 referencia
35        public DateTime LastLoginDate { get; set; }
36    }
37 }
```

MVC – Servicios

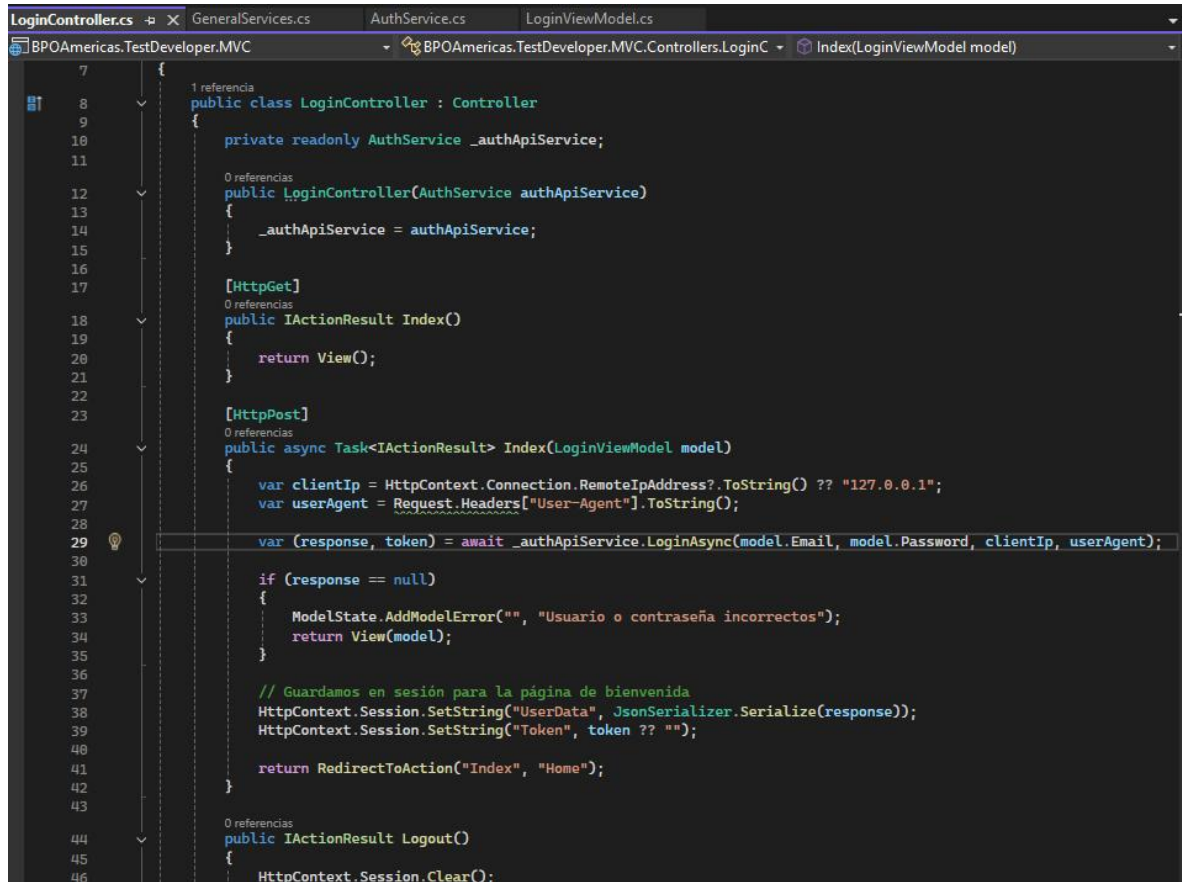
Realizamos el servicio de Autenticación, instanciando httpClient para poder apuntar a nuestra API y enviándole el mismo objeto request para que este nos lea de manera adecuada, siempre encriptando de manera adecuada



```
1  using BPOAmericas.TestDeveloper.MVC.Models.Login;
2  using System.Text;
3
4  namespace BPOAmericas.TestDeveloper.MVC.Services.Auth
5  {
6      6 referencias
7      public class AuthService
8      {
9
10         0 referencias
11         public AuthService(HttpClient httpClient)
12         {
13             _httpClient = httpClient;
14         }
15
16         public async Task<LoginApiResponse response, string token>
17             1 referencia
18             LoginAsync(string email, string password, string clientId, string userAgent)
19         {
20             var request = new LoginApiRequest
21             {
22                 UserName = Convert.ToBase64String(Encoding.UTF8.GetBytes(email)),
23                 UserPassword = Convert.ToBase64String(Encoding.UTF8.GetBytes(password)),
24                 ClientIP = clientId,
25                 UserAgent = userAgent
26             };
27
28             var httpResponse = await _httpClient.PostAsJsonAsync("Security/LoginUser", request);
29
30             if (!httpResponse.IsSuccessStatusCode)
31                 return (null, null);
32
33             var loginResponse = await httpResponse.Content.ReadFromJsonAsync<LoginApiResponse>();
34
35             string token = null;
36             if (httpResponse.Headers.TryGetValues("Authorization", out var values))
37                 token = values.FirstOrDefault();
38
39             return (loginResponse, token);
40         }
41     }
```

MVC - Controlador

Luego de la creación de nuestro servicio nos vamos a la creación del controlador, en el que hacemos la parte del llamado y obtenemos de la vista el email y la contraseña, instanciamos la clase de auth y generamos un RedirectToAction a el controlador Home que solo nos va a mostrar lo que hay en el Objeto Response



```
7
8
9
10
11
12
13
14
15
16
17
18
19
20
21
22
23
24
25
26
27
28
29
30
31
32
33
34
35
36
37
38
39
40
41
42
43
44
45
46

1 referencia
public class LoginController : Controller
{
    private readonly AuthService _authApiService;

    0 referencias
    public LoginController(AuthService authApiService)
    {
        _authApiService = authApiService;
    }

    [HttpGet]
    0 referencias
    public IActionResult Index()
    {
        return View();
    }

    [HttpPost]
    0 referencias
    public async Task<IActionResult> Index(LoginViewModel model)
    {
        var clientIp = HttpContext.Connection.RemoteIpAddress?.ToString() ?? "127.0.0.1";
        var userAgent = Request.Headers["User-Agent"].ToString();

        var (response, token) = await _authApiService.LoginAsync(model.Email, model.Password, clientIp, userAgent);

        if (response == null)
        {
            ModelState.AddModelError("", "Usuario o contraseña incorrectos");
            return View(model);
        }

        // Guardamos en sesión para la página de bienvenida
        HttpContext.Session.SetString("UserData", JsonSerializer.Serialize(response));
        HttpContext.Session.SetString("Token", token ?? "");

        return RedirectToAction("Index", "Home");
    }

    0 referencias
    public IActionResult Logout()
    {
        HttpContext.Session.Clear();
    }
}
```

MVC – VISTA

El response esta en la vista Home

```
Index.cshtml  LoginController.cs  GeneralServices.cs  AuthService.cs  LoginViewModel.cs
BPOAmericas.TestDeveloper.MVC
1  @model BPOAmericas.TestDeveloper.MVC.Models.Login.LoginApiResponse
2
3  @{
4      ViewData["Title"] = "Bienvenida";
5  }
6
7  <div class="container mt-5" style="max-width: 500px;">
8      <h2 class="mb-4 text-center">Bienvenido</h2>
9
10     <div class="mb-3">
11         <label class="form-label">ID Usuario</label>
12         <input type="text" class="form-control" value="@Model.IdUser" readonly />
13     </div>
14
15     <div class="mb-3">
16         <label class="form-label">Nombre de Usuario</label>
17         <input type="text" class="form-control" value="@Model.UserName" readonly />
18     </div>
19
20     <div class="mb-3">
21         <label class="form-label">Perfil</label>
22         <input type="text" class="form-control" value="@Model.UserProfile" readonly />
23     </div>
24
25     <div class="mb-3">
26         <label class="form-label">Último Ingreso</label>
27         <input type="text" class="form-control" value="@Model.LastLoginDate" readonly />
28     </div>
29
30     <form asp-controller="Login" asp-action="Logout" method="get">
31         <button type="submit" class="btn btn-danger w-100">Cerrar Sesión</button>
32     </form>
33 </div>
```

Vista del proyecto terminado

Y así quedaría el front:

BPOAmericas.TestDeveloper.MVC Home Privacy

Iniciar Sesión

Usuario (correo)

Contraseña

Iniciar Sesión

© 2026 - BPOAmericas.TestDeveloper.MVC - [Privacy](#)

BPOAmericas.TestDeveloper.MVC Home Privacy

Bienvenido

ID Usuario

5

Nombre de Usuario

TEST USER

Perfil

lowlevel

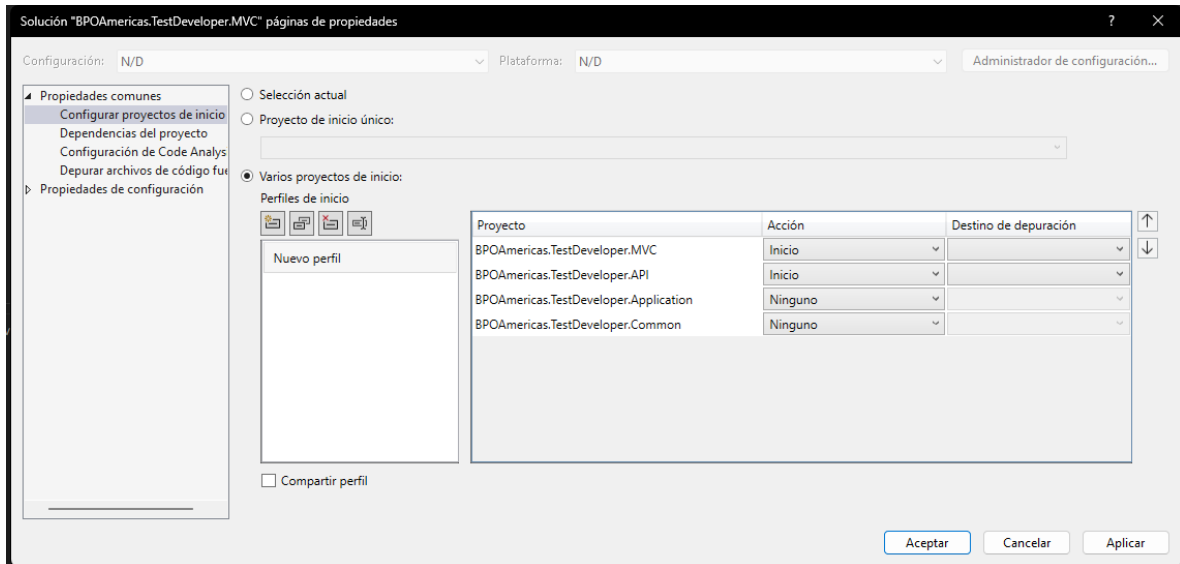
Último Ingreso

28/09/2026 8:39:38 p. m.

Cerrar Sesión

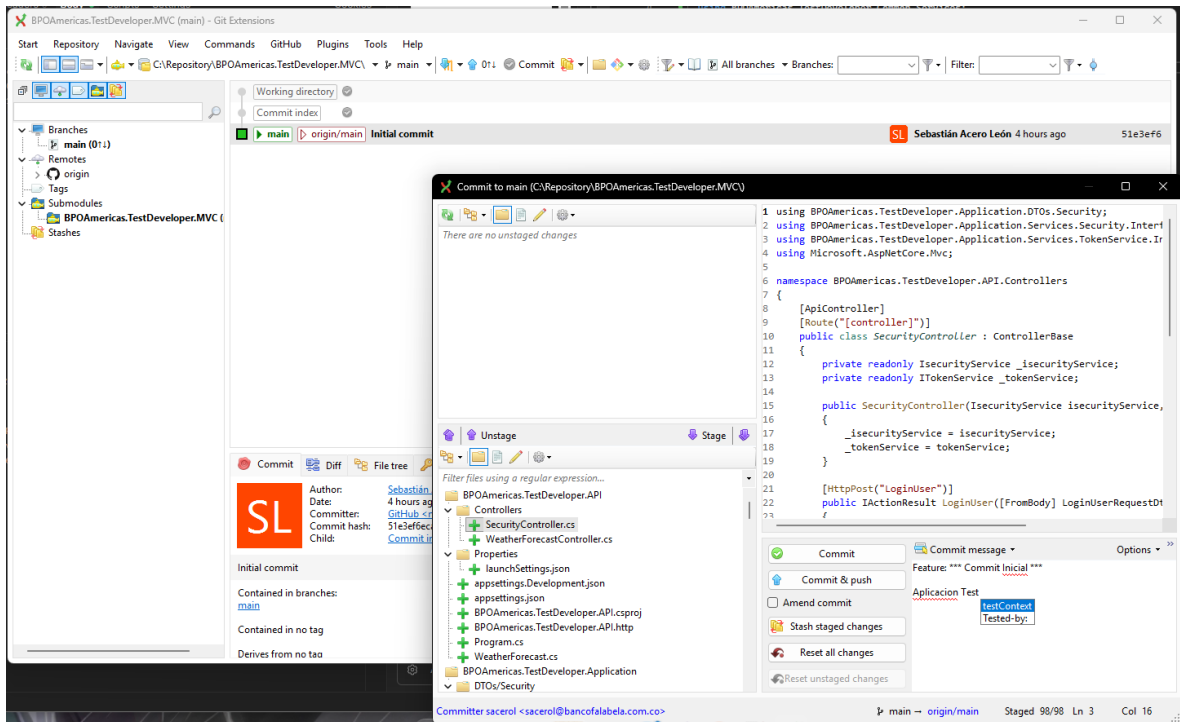
© 2026 - BPOAmericas.TestDeveloper.MVC - [Privacy](#)

NOTA: Se deben ejecutar los dos proyectos al mismo tiempo, ya que el API corre por un lado y el MVC corre por otro



Subir cambios al repositorio

Por ultimo hacemos commit y push a la solución creada para que esta suba al repositorio y poderlo hacer visible y no mas en local



Recargamos el GITHUB para ver que el proyecto este completo y con todos sus módulos y carpetas creadas y subidas correctamente

Walax11 / BPOAmericas.TestDeveloper.MVC

CodeIssuesPull requestsAgentsActionsProjectsWikiSecurity and qualityInsightsSettings

BPOAmericas.TestDeveloper.MVCPublic

PinWatchForkStar

main1 Branch0 TagsGo to fileAdd fileCode

sacerolFeature: *** Commit Inicial ***612773d · now2 Commits

BPOAmericas.TestDeveloper.API	Feature: *** Commit Inicial ***	now
BPOAmericas.TestDeveloper.Application	Feature: *** Commit Inicial ***	now
BPOAmericas.TestDeveloper.Common	Feature: *** Commit Inicial ***	now
BPOAmericas.TestDeveloper.MVC	Feature: *** Commit Inicial ***	now
.gitignore	Initial commit	4 hours ago

README

Add a README

Help people interested in this repository understand your project.

About

No description, website, or topics provided.

Activity0 stars0 watching0 forks

Releases

No releases published
[Create a new release](#)

Packages

No packages published
[Publish your first package](#)

Contributors

Walax11 Sebastián Acero León

Languages

CF67.1%HTML26.4%CSS3.5%JavaScript1%